



5.1. Common Problems on Recursion

"Recursion is when a function calls itself, until it doesn't."

Main Section:

 **5. Recursion**

Contents:

1. Fibonacci

2. The Rabbit Problem

3. Recursive Problems

Power (Exponentiation)

Factorial Number

Palindrome Checking

Maximum Element from a List

Reversing the Digits of a Number

Sum of the First n Natural Numbers

Sum of Digits of a Number

Sum of Elements of a List

List Having a Minimum Sum

Longest Common Subsequence (LCS)

Summary

References

Recursion becomes much easier to understand once you see it solving actual problems. Many famous algorithms in Computer Science are recursive by nature because their solutions can be reduced into smaller versions of the same problem.

In this section, we will explore some of the most iconic recursive problems. Some of them may already be familiar to you—but this time, recursion will be the one doing the work.

And naturally, one of the best places to begin is with the most famous recursive sequence in programming: Fibonacci.

1. Fibonacci

One of the most famous problems in programming and algorithmic studies is the Fibonacci sequence. The Fibonacci sequence is a sequence of numbers where each number is the sum of the two preceding numbers.

The sequence begins as follows:

0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, ...

Notice the pattern carefully:

- $1 + 1 = 2$
- $1 + 2 = 3$
- $2 + 3 = 5$
- $3 + 5 = 8$

And the process continues forever.

Mathematically, Fibonacci is naturally recursive because every value depends on smaller Fibonacci values. The recursive definition becomes:

$$F(n) = F(n - 1) + F(n - 2)$$

And immediately, recursion becomes the perfect tool for solving it.

Finding the 18th Fibonacci Number

Suppose we want to find: $F(18)$

For this recursive version, we define the base case as:

- If $n = 1$ or $n = 2$, return 1 (We will ignore 0 in this specific implementation.)

So the recursive process works like this:

- $F(18) = F(17) + F(16)$
- $F(17) = F(16) + F(15)$
- $F(16) = F(15) + F(14)$

And recursion continues repeatedly. Notice something important:

■ ***The same function keeps calling itself again and again with smaller values.***

The function does not instantly calculate the answer. Instead, every recursive call pauses and waits for smaller recursive calls beneath it to finish first.

Eventually, recursion reaches the base cases:

- $F(2) = 1$
- $F(1) = 1$

And this is where the recursive chain finally stops growing. Now the recursive calls begin returning upward through the call stack.

Returning Back Up the Recursive Chain

Once the base cases are reached, the unfinished recursive calls finally begin computing their results:

- $F(3) = F(2) + F(1) = 1 + 1 = 2$
- $F(4) = F(3) + F(2) = 2 + 1 = 3$
- $F(5) = F(4) + F(3) = 3 + 2 = 5$
- $F(6) = F(5) + F(4) = 5 + 3 = 8$

And the process continues upward:

- $F(7) = 13$
- $F(8) = 21$
- $F(9) = 34$

- $F(10) = 55$
- $F(11) = 89$
- $F(12) = 144$
- $F(13) = 233$
- $F(14) = 377$
- $F(15) = 610$
- $F(16) = 987$
- $F(17) = 1597$

Finally: $F(18) = 2584 \rightarrow$ So the 18th Fibonacci number is: 2584

Understanding the Recursive Workflow Carefully

The Fibonacci problem is one of the best demonstrations of recursion because it clearly shows how recursive calls depend on each other.

Here is the important idea. When we call:

```
fibonacci(18)
```

the function immediately splits into:

```
fibonacci(17) + fibonacci(16)
```

But to calculate `fibonacci(17)`, the function must again calculate:

```
fibonacci(16) + fibonacci(15)
```

And to calculate `fibonacci(16)`:

```
fibonacci(15) + fibonacci(14)
```

This continues repeatedly until eventually the smallest problems are reached:

```
fibonacci(2)
fibonacci(1)
```

These are the base cases. Only after the base cases return values can the larger recursive calls finally compute their own answers.

So recursion works in two phases:

- **Phase 1 — Expanding Downward:** The function keeps calling itself with smaller values.
- **Phase 2 — Returning Upward:** Once the base cases are reached, results return upward and combine together.

This is why recursion behaves like a chain reaction. Every larger solution depends on smaller solutions finishing first.

Time Complexity of Recursive Fibonacci

Although recursive Fibonacci is elegant, it has a major weakness. It repeats the same calculations many times.

For example: $F(18)$ needs $F(17)$ and $F(16)$. But $F(17)$ itself also needs $F(16)$

This means: $F(16)$ gets recomputed repeatedly. Many smaller Fibonacci values are recalculated again and again. This duplicated work grows rapidly as n becomes larger.

A rough upper bound for recursive Fibonacci is: $O(2^n)$

However, the tighter upper bound is related to the famous Golden Ratio:

$$\varphi = \frac{1 + \sqrt{5}}{2} \approx 1.618$$

So the tighter complexity becomes: $O(\varphi^n)$, where: $\varphi \approx 1.618$. This explains why recursive Fibonacci becomes extremely slow for large values of n . And later on, this exact weakness becomes one of the motivations behind:

Memoization and Dynamic Programming

Program

```
def fibonacci(n):
    if n == 1 or n == 2:
        return 1
    else:
        return fibonacci(n - 1) + fibonacci(n - 2)
```

The main function:

$n = 18 \rightarrow \text{fib} = \text{fibonacci}(18)$

Output: 2584

Workflow of the Code

Let us carefully observe how the recursion behaves inside the program.

- When the user enters: 18 $\rightarrow$ the program calls: `fibonacci(18)`
- Since 18 is not the base case, the function returns: `fibonacci(17) + fibonacci(16)`

But those functions are also not base cases, so they again call smaller versions of themselves. This recursive expansion continues deeper and deeper:

```
fibonacci(5)
→ fibonacci(4) + fibonacci(3)

fibonacci(4)
→ fibonacci(3) + fibonacci(2)
```

Eventually: `fibonacci(2)` and `fibonacci(1)` are reached. These are the stopping points.

Now the recursive calls finally begin returning values upward:

```
fibonacci(3)
= 1 + 1
= 2

Then:
fibonacci(4)
= 2 + 1
= 3

Then:
fibonacci(5)
```

```
= 3 + 2  
= 5
```

And this process continues upward until finally:

```
fibonacci(18)  
= 2584
```

So the recursive function continuously breaks the problem into smaller subproblems, waits for the smaller problems to finish, then combines the returned results together. This is recursion in its purest form.

2. The Rabbit Problem

Interestingly, the Fibonacci sequence was not originally introduced as just a mathematical curiosity. It came from a story.

One of the earliest and most famous explanations of Fibonacci appears in a rabbit population problem described by the Italian mathematician Leonardo Fibonacci. At first glance, the problem sounds simple. But underneath it hides one of the clearest demonstrations of recursive growth.

The problem is described like this:

Suppose we begin with one pair of rabbits. A rabbit pair does not reproduce during its first two months of life. After reaching maturity, the pair produces one new pair of rabbits every month. Every newly born pair follows the exact same rule.

Immediately, recursion appears. Because every new rabbit pair behaves exactly like the original pair. The same process repeats again and again.

Observing the Growth Month by Month

Let us carefully observe what happens.

- **Month 1:** There is only 1 pair of rabbits. The rabbits are too young to reproduce.
- **Month 2:** Still 1 pair of rabbits. The pair has not reached reproductive maturity yet.

- **Month 3:** Now the original pair becomes mature and produces another pair. So we now have: 2 pairs of rabbits
- **Month 4:** The original pair produces again. The younger pair is still too young to reproduce. So we now have 3 pairs of rabbits
- **Month 5:** Now things become interesting. The original pair reproduces. The second pair has also matured and reproduces. So the total becomes 5 pairs of rabbits
- **Month 6:** Now three mature pairs reproduce first pair reproduces, second pair reproduces, third pair reproduces. The total becomes 8 pairs of rabbits

And the process continues: 1, 1, 2, 3, 5, 8, 13, 21, 34, ... Which is exactly the Fibonacci sequence.

Why This Becomes Recursive

The Rabbit Problem perfectly demonstrates recursive thinking because the number of rabbits in the current month depends entirely on the previous two months.

For example:

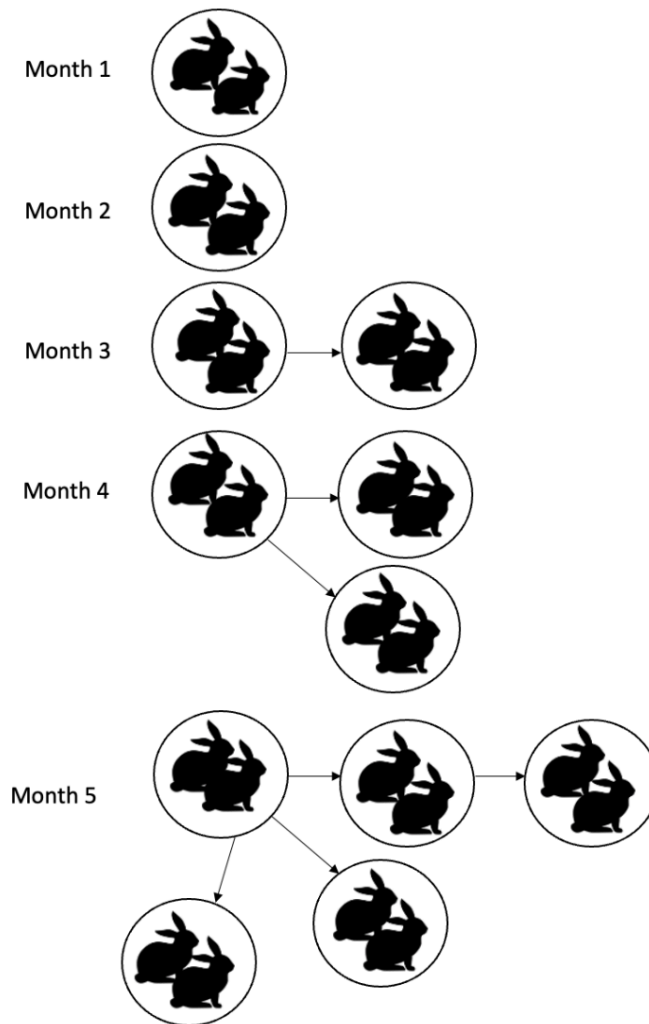
- Month 6 depends on Month 5 and Month 4
- Month 7 depends on Month 6 and Month 5
- Month 8 depends on Month 7 and Month 6

Mathematically, this becomes: $F(n) = F(n - 1) + F(n - 2)$

And this recursive relationship appears naturally from the rabbit reproduction rules themselves. Why? Because rabbits already alive last month continue existing, new rabbits are produced by mature rabbits from earlier months. So every month becomes the combination of the previous two states.

Visualizing the Recursive Structure

The Rabbit Problem is often represented as a recursive tree.



Each node represents the rabbit population of a certain month. And each node branches into the populations from the previous two months.

For example: $F(6) = F(5) + F(4)$

- Then $F(5)$ further expands into: $F(5) = F(4) + F(3)$
- And $F(4)$ expands into $F(4) = F(3) + F(2)$
- The recursive tree continues expanding downward until eventually reaching the base cases: $F(1) = 1$ and $F(2) = 1$
- And then the recursive results begin returning upward again.

This is why the Rabbit Problem is such an important example in recursion. It demonstrates that recursion is not just a programming technique. It is a way of modeling growth itself. Simple rules repeated over time can generate

surprisingly complex behavior. And the Fibonacci sequence is one of the earliest and most elegant examples of that idea.

3. Recursive Problems

Recursion can solve a surprisingly large number of problems. Some problems are mathematical, some involve strings or lists, while others are related to optimization and dynamic programming.

In this section, we will briefly introduce several common recursive problems and observe how recursion approaches each one. The purpose here is not only to memorize solutions, but to recognize an important pattern:

Most recursive problems can be reduced into smaller versions of themselves.

And once that recursive relationship is identified, the algorithm almost “builds itself”.

The following problems are some of the most common recursive exercises in programming and algorithmic studies.

Power (Exponentiation)

Exponentiation is one of the simplest recursive problems. To calculate:

$$a^b = a \times a^{b-1}$$

the algorithm repeatedly reduces the exponent until eventually reaching the base case: $a^0 = 1$.

Recursion works naturally here because every power depends on a smaller power.

Code:

```
def power(a, b):
    if b == 0:
        return 1
    else:
        return a * power(a, b - 1)
```

a, b = 2, 6 → Output: 64

Factorial Number

Factorial is another classic recursive problem. The recursive relationship becomes:

$$n! = n \times (n - 1)!$$

The recursion continues until reaching: $1! = 1$

Factorial is often used to teach recursion because the mathematical definition itself is already recursive.

Code:

```
def factorial(n):
    if n == 1:
        return 1
    else:
        return n * factorial(n - 1)
```

n = 7 → Output: 5040

Greatest Common Divisor (GCD) and Lowest Common Multiple (LCM)

Recursion is heavily used in the Euclidean Algorithm for finding the Greatest Common Divisor. The recursive relationship is:

$$GCD(a, b) = GCD(b, a \bmod b)$$

The process repeatedly reduces the problem until the remainder becomes zero. Once the GCD is known, the Lowest Common Multiple can also be calculated easily. This is one of the most efficient recursive algorithms in mathematics.

Code:

```
def greatest_common_divisor(n1, n2):
    if n1 == 0:
        return n2
    elif n2 == 0:
        return n1
    else:
        return greatest_common_divisor(n2, n1 % n2)
```

n1, n2 = 12, 16 → Output: 4

```
def lowest_common_multiple(n1, n2):
    if n1 == 0 or n2 == 0:
        return 0
    else:
        return (n1 * n2) // greatest_common_divisor(n1, n2)
```

n1, n2 = 12, 14 → Output: 84

Palindrome Checking

A palindrome is a word, phrase, or number that reads the same forward and backward. Examples: ***"level"***, ***"radar"***, ***"madam"***, ***121, 232***

Recursion solves this problem for numbers by:

1. Comparing the first and last digit
2. Removing them
3. Recursively checking the smaller remaining digits

The recursion stops once the number becomes empty, or only one digit remains. This demonstrates how recursion naturally reduces problems layer-by-layer.

Code:

```
def check_palindrome(n, temp):  
    if n == 0:  
        return temp  
    temp = (temp * 10) + (n % 10)  
  
    return check_palindrome(n // 10, temp)
```

232 → Output: True

Maximum Element from a List

Recursion can also search for the maximum value in a list. The idea is simple: Compare the current element with the maximum of the remaining list, return the larger value.

The list continuously becomes smaller until only one element remains. That final element becomes the base case.

Code:

```
def find_max(L):  
    if len(L) == 1:  
        return L[0]  
    else:  
        rest = find_max(L[1:])  
        if L[0] > rest:  
            return L[0]  
        else:  
            return rest
```

L = [12, 11, 19, 28, 13] → Output: 28.0

Reversing the Digits of a Number

Recursion can reverse numbers by repeatedly extracting digits. For example:

- Take the last digit
- Recursively process the remaining number
- Combine the digits in reverse order

The recursive calls continue until the number becomes zero. This problem demonstrates recursive decomposition very clearly.

```
def digits(n): # This function return the number of digits
    d = 1
    while n // 10 != 0:
        d += 1
        n = n // 10
    return d

def reverse_number(n):
    if digits(n) == 1:
        return n
    else:
        num = n // 10
        rev = reverse_number(num)
        dev = digits(rev)

        temp = (n % 10) * (10 ** dev) + rev
    return temp
```

n = 7749 → Output: 9477

Sum of the First n Natural Numbers

This is one of the simplest recursive mathematical problems. The recursive definition becomes:

$$S(n) = n + S(n - 1)$$

with the base case: $S(1) = 1$

The function repeatedly reduces the number until the smallest case is reached.

```
def sum_natural_number(n):  
    if n == 0:  
        return 0  
    elif n == 1:  
        return 1  
    else:  
        return n + sum_natural_number(n - 1)
```

n = 12 → Output: 78

Sum of Digits of a Number

Recursion can also calculate the sum of digits.

For example: Separate the last digit, add it to the recursive sum of the remaining digits

- $1234 = 4 + 123$
- Then: $123 = 3 + 12$
- And so on until the number becomes zero.

```
def sum_digits(n):  
    if n == 0:  
        return 0  
    return n % 10 + sum_digits(int(n / 10))
```

n = 123456 → Output: 21

Sum of Elements of a List

This problem recursively adds all elements inside a list. The recursive strategy becomes:

- Add the first element
- Recursively calculate the sum of the remaining list

Eventually, the list becomes empty, which becomes the base case. This problem closely resembles recursive summation in mathematics.

```
def find_sum(s):  
    if len(s) == 1:  
        return s[0]  
    else:  
        total = s[0] + find_sum(s[1:])  
        return total
```

s = [12, 11, 19, 28, 13, 10] → Output: 93

List Having a Minimum Sum

Recursion can also compare multiple lists or sublists to determine which one has the minimum total sum.

The algorithm recursively computes the sums of smaller structures and compares them step-by-step. This type of recursive thinking becomes important in optimization problems later on.


```

def list_sum(lst): # This function calculates the sum of each list
    if len(lst) == 1:
        return lst[0]
    return lst[0] + list_sum(lst[1:])

def minimum_sum_list(n_lst):
    if len(n_lst) == 1:
        return n_lst[0]

    current_sum = list_sum(n_lst[0])
    next_list = minimum_sum_list(n_lst[1:])
    next_sum = list_sum(next_list)

    if current_sum < next_sum:
        return n_lst[0]
    else:
        return next_list

```

n_lst: [[1.0, 2.0, 4.0, 5.0], [3.0, 2.0, 7.0], [1.0, 3.0, 4.0, 9.0, 2.0]] → Output: [3.0, 2.0, 7.0]

Longest Common Subsequence (LCS)

The Longest Common Subsequence problem is one of the most famous recursive problems in algorithm design. The objective is to find the longest sequence shared between two strings while maintaining order.

Recursion solves LCS by comparing characters recursively:

- If characters match: Include the character → Continue recursively
- If characters do not match → Explore both possibilities recursively, return the longer result

Although recursive LCS is elegant conceptually, it becomes computationally expensive due to repeated calculations.

This problem later becomes one of the most important examples of Dynamic Programming.

```
def lcs(s1, s2, m, n):
    if m == 0 or n == 0:
        return ""
    elif s1[m - 1] == s2[n - 1]:
        return lcs(s1, s2, m - 1, n - 1) + s1[m - 1]
    else:
        lcs1 = lcs(s1, s2, m, n - 1)
        lcs2 = lcs(s1, s2, m - 1, n)
        if len(lcs1) > len(lcs2):
            return lcs1
        else:
            return lcs2
```

s1 = 12929393

s2 = 2191922

Output: 122

These problems only represent the beginning of recursion. Once recursion becomes familiar, many more advanced recursive problems become easier to understand, including Tree traversal; Graph exploration; Maze solving; Backtracking; Divide and Conquer; Dynamic Programming; Artificial Intelligence search systems

Recursion appears almost everywhere in algorithmic studies because many problems naturally describe themselves recursively. And the best way to truly understand recursion is practice.

There are countless recursive exercises and challenges available online. Simply searching *"Recursion Practice"*; *"Recursive Exercises"*; *"Recursive Programming Problems"* will reveal hundreds of additional problems to explore.

Because recursion is not learned by memorization alone. It is learned by repeatedly thinking recursively.

Summary

Throughout these recursive problems, you have learned that recursion is not merely an algorithmic technique—it is a versatile and almost majestic way of solving problems through self-reference and reduction.

From simple mathematical computations like factorials, exponentiation, and Fibonacci, recursion gradually expands into far more sophisticated territories in Computer Science. It becomes the foundation behind traversal systems, searching mechanisms, optimization problems, and even intelligent exploration used in Artificial Intelligence.

More importantly, recursion does not exist alone. It serves as the backbone of major algorithmic methodologies such as Divide and Conquer, Dynamic Programming, and Backtracking, where complex problems are continuously reduced into smaller, more manageable forms.

And this is what makes recursion so powerful:

A single idea—solving smaller versions of the same problem—can scale from basic exercises to some of the most advanced systems in Computer Science.

References

<https://www.geeksforgeeks.org/dsa/program-for-nth-fibonacci-number/>

<https://www.geeksforgeeks.org/dsa/recursion-practice-problems-solutions/>

<https://leetcode.com/problem-list/recursion/>

<https://www.youtube.com/watch?v=ye1kMmRtlw8>